DAY 18

Database Connectivity & CRUD API

Build a persistent Student API with Django, Django REST Framework, and SQLite.

Learning goals

Topics: Django models • Django ORM • migrations • serializers • API views or viewsets • CRUD • database configuration

By the end, you will have a Student record stored in a database and API routes to create, read, update, and delete records.

1. Set up the project

Activate your virtual environment, install the packages, then create an app if you do not already have one.

```
python -m pip install django djangorestframework
django-admin startproject config .
python manage.py startapp students
```

In config/settings.py, add the app and REST framework:

```
INSTALLED_APPS = [
    # ... default Django apps
    'rest_framework',
    'students',
]
```

Database choice: SQLite is configured by default and is a good starting point. Django creates the local database file when migrations are applied. For PostgreSQL or MySQL, install the matching Python database driver and set the ENGINE, NAME, USER, PASSWORD, HOST, and PORT in DATABASES.

2. Define the Student model

Create students/models.py. Django fields define the data types and constraints that the ORM stores in the database.

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    age = models.PositiveSmallIntegerField()

    def __str__(self):
        return self.name
```

Create and apply migrations. A migration records a schema change; migrate applies it to the configured database.

```
python manage.py makemigrations students
python manage.py migrate
```

3. Serialize model data

Create students/serializers.py. ModelSerializer maps model fields to JSON and validates incoming data.

```
from rest_framework import serializers
from .models import Student

class StudentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Student
        fields = ["id", "name", "email", "age"]
```

4. Add CRUD API routes

A ModelViewSet provides list, create, retrieve, update, partial update, and delete actions. Add students/views.py:

```
from rest_framework.viewsets import ModelViewSet
from .models import Student
from .serializers import StudentSerializer

class StudentViewSet(ModelViewSet):
    queryset = Student.objects.all().order_by("id")
    serializer_class = StudentSerializer
```

Create students/urls.py and connect a router. Then include it from the project URL configuration.

```
from rest_framework.routers import DefaultRouter
from .views import StudentViewSet

router = DefaultRouter()
router.register("students", StudentViewSet)
urlpatterns = router.urls
```

```
# config/urls.py
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("api/", include("students.urls")),
]
```

5. Run and try the API

```
python manage.py runserver
```

Open **http://127.0.0.1:8000/api/students/** in a browser to see the browsable API. The API root lists records; use Postman or another API client for methods beyond GET.

Action	Method and URL	Example request body
List students	GET /api/students/	—
Create student	POST /api/students/	{"name":"Asha Rao","email":"asha@example.com","age":20}
Read one	GET /api/students/1/	—
Update all fields	PUT /api/students/1/	Send all required fields
Update some fields	PATCH /api/students/1/	{"age":21}
Delete	DELETE /api/students/1/	—

Use JSON as the request body and set Content-Type: application/json. A successful create typically returns 201; reads return 200; a successful delete returns 204. Duplicate email or invalid data returns a validation error.

Expected output

A working Django application with migrations applied, database connectivity established, and a Student CRUD REST API. Confirm by creating a student, fetching it, changing a field, and deleting it.

Quick troubleshooting

• “no such table”: run makemigrations and migrate.
• 404: check the api/ prefix, router registration, and trailing slash.
• 400: inspect serializer errors and send valid JSON.
• Database driver error: confirm the chosen DB driver is installed and DATABASES settings match the database.